

實作段講義 | AI 金融應用

國立成功大學暑期微學分課程 2026 年 8 月 18–19 日

本講義為課程公開倉庫的合訂本，最新版本請見：
github.com/csdm67931/AI-4

本課程為教育性內容，不構成任何投資建議。

課程架構 | 兩天七小時

每天上午 9:00–12:30 (3.5 小時) = 演講 1.5 小時 + AI 實作 2 小時。

休息時間由講者視情況插入各段落中。

全課地圖

Day 1 (3.5h)	Day 2 (3.5h)
└─ 演講 1.5h：川普為什麼課關稅	└─ 演講 1.5h：美中攻防與版圖的未來
① 川普要什麼	① 對中國怎麼出手
② 工具怎麼選	② 版圖往哪走
③ 誰付錢	③ 怎麼檢查一個說法
④ 股市與匯率怎麼反應	④ 帶走五樣東西
└─ 實作 2h：開工	└─ 實作 2h：完工與報告
└─ 觀念與環境（六件套）	└─ 從 MEMORY 接手
└─ 拷問 → PLAN／TASK／MEMORY	└─ 完成關卡三～五
└─ 關卡一、二（抓、理）	└─ 分組報告

Day 1 實作段 (120 分)

時段	分	內容
0:00–0:05	5	分組、抽題
0:05–0:15	10	核心觀念三頁：成果存倉庫才帶得走 / 三個檔案 / 動手前先拷問
0:15–0:45	30	環境建置 (正式單元)：裝 Python → 裝 Antigravity → Gmail 登入 → <code>pip install</code> → 連 GitHub → 跑出第一支程式
0:45–0:53	8	講者示範一次完整拷問流程

時段	分	內容
0:53–1:10	17	各組實作拷問 → 產出 <code>PLAN.md</code> 、 <code>TASK.md</code> 、 <code>MEMORY.md</code>
1:10–1:50	40	實作關卡一、二；每過一關 commit 一次
1:50–2:00	10	收尾儀式：push + 寫 <code>SETUP.md</code> / <code>requirements.txt</code> + 在 <code>MEMORY</code> 寫「明天第一步」

第一小時的過關條件（五樣都有就過關）

1. 倉庫開好、組員都進得去
2. 環境跑得出第一支程式
3. `PLAN.md` 六節齊全，存進倉庫
4. `TASK.md` 第一張寫好（目標、材料、驗收）
5. `MEMORY.md` 開檔，記下被 AI 問倒的洞

Day 2 實作段（120 分）

時段	分	內容
0:00–0:12	12	照自己的 <code>SETUP.md</code> 重建環境（文件驗收）+ 從 <code>MEMORY.md</code> 接手
0:12–0:46	34	檢驗與修改：完成關卡三、四、五，用四判準自檢
0:46–0:48	2	換場、報告順序抽籤
0:48–2:00	72	分組報告（每組 5–8 分鐘）

五個關卡（每組都走同一副骨架，只換原料）

關卡	要交出什麼	怎麼算過
一、抓	用程式抓到原始資料，原封不動存檔	檔案存在、說得出網址與抓取時間
二、理	清洗與對齊：日期統一、數值可運算	隨機抽一天，數字對得上來源
三、看	畫圖，並標上至少 2 個關稅事件日	圖上事件日清楚可指
四、監控化	做成可重跑的工具，再執行一次會更新到最新	台上現場重跑一次
五、結論報告	讓 AI 讀你自己的資料，產出四節結論報告	每個數字都能在自己的資料裡指出來

走 EvoCode 路徑的組別，關卡一、二由平台內建報價取代，關卡四改為「回測面板跑得動」。詳見 [reference/EvoCode速查.md](#)。

分組報告（每組 5–8 分鐘）

段	內容
題目與資料	抽到什麼、資料哪裡來

段	內容
過程與困難	卡在哪（講真話——被限速、格式對不齊、AI 生的程式跑不動都算）
解決辦法	怎麼繞過去的；AI 哪裡講錯、你們怎麼發現的
目前成果	現場重跑一次工具 + 展示結論報告
資料限制	這份資料哪裡不能信

素材不用現場編——MEMORY.md 打開就是。

評分構面

構面	權重	看什麼
過程與困難的真實度	30%	講得出具體卡點，不是「一切順利」
解決方法	25%	有自己的判斷，不是全程照抄
成果可重跑	25%	台上重跑成功；失敗但能解釋原因也給分
資料限制與誠實度	20%	說得出限制；用四判準自檢過

開課前準備 | 帳號申請

現場只登入、不註冊。十八個人同時辦帳號會吃掉半堂課，所以務必在課前完成。
辦好之後回來對照本頁的「檢查方式」，確認能登人才算完成。

總覽

帳號	用途	需要等信嗎	課前必辦
Google 帳號	登入 Antigravity (本機開發工具)、使用 AI 助手	不用	是
GitHub 帳號	存放兩天的成果	不用 (但要驗證信箱)	是
群益期貨模擬帳號 (QIT)	看國內期貨與選擇權的實際報價	要，等 Email	是，務必提早辦
群益外匯美股模擬帳號 (CTA)	看外匯與海外商品報價、使用 EvoCode	要，等 Email	是，務必提早辦

兩個群益帳號是不同平台：期貨走 QIT，外匯與美股走 CTA (EvoCode 在 CTA 上)。
兩個都申請，兩天課程都會用到。

一、Google 帳號

為什麼要辦：這兩天在本機用 Antigravity (或 Cursor) 這套 IDE 寫程式，登入方式是 Google 帳號；IDE 裡的 AI 助手也是同一個帳號授權使用。

怎麼辦：

1. 已有 Gmail 或其他 Google 帳號的同學可以直接用，不用另外申請。
2. 沒有的話到 accounts.google.com/signup 申請一個新帳號。

辦好的檢查方式：

1. 能用這個帳號登入 accounts.google.com。
2. 記得帳號密碼，開課當天登入 Antigravity 會用到 (詳見 [02-環境設定.md](#))。

二、GitHub 帳號

為什麼要辦：兩天做出來的東西 (程式、資料、報告) 都要存進 GitHub 倉庫才帶得走——教室電腦有還原系統，下課就會清空，存在電腦桌面等於白做。GitHub 帳號是這兩天成果的家。

怎麼辦：

1. 到 github.com/signup 註冊。
2. 需要一個可以收信的信箱 (會寄驗證信，務必完成信箱驗證這一步)。
3. 註冊過程有人機驗證，跟著畫面指示做完即可。
4. 使用者名稱建議用真實姓名或學號相關的英文 / 數字組合，方便組員與老師辨認。

辦好的檢查方式：

1. 能用剛註冊的帳號登入 github.com。
2. 登入後右上角看得到自己的頭像與使用者名稱。

課堂上第一個實作動作就是「組長開一個新倉庫、把組員加進去」，詳細步驟見 [02-環境設定.md](#)。

三、群益模擬帳號 (兩個都要辦)

為什麼要辦：這門課的特色是與產業實務結合。你會用**真實的券商報價**做出每天可以跑的觀察工具，而不是只看歷史資料寫報告。模擬帳號讓你看到實際的商品報價與交易介面，**不會用到真的錢**，純粹是教學與資料存取用途。

兩個平台分工不同，**兩個都要申請：**

3-1 期貨模擬帳號 (QIT 平台)

國內期貨與選擇權報價。申請網址：

<https://www.capitalfutures.com.tw/zh-tw/simulatedtrading/qit?empno=a97157>



申請流程 (三步驟)

1. 填姓名、電子郵件、手機號碼、圖形驗證碼，勾選同意條款後送出。
2. 完成手機簡訊驗證後，**帳號與密碼會寄到你填的信箱**。
3. 下載 QIT 平台 (電腦版 / iOS / Android 都有)，用收到的帳密登入。

3-2 外匯美股模擬帳號 (CTA 平台，EvoCode 在這裡)

外匯、海外指數、美股 CFD 報價，以及 **EvoCode** (用 C# 寫自訂技術指標的環境)。

申請網址：

<https://cfmt.capitalfutures.com.tw/zh-TW/DemoAccount/Apply?sid=A97157#reg>



申請流程

1. 填寫申請表，完成簡訊驗證，勾選個資同意書後送出。
2. **至信箱收取交易帳號**。
3. 安裝群益交易王 CTA 電腦版 (見 [02-環境設定.md](#))，用收到的帳密登入。

檢查方式 (兩個帳號都要確認)

1. 信箱收到群益寄來的帳號通知信，裡面有帳號與密碼 (或設定密碼的連結)。
2. 能用這組帳號登入對應的平台，看得到商品報價在跳動。

⚠ 這兩個申請都要**等 Email**，不是填完就有。
現場才辦會等不到信，整堂課會卡住——**請務必課前辦好**。

帳號都辦好之後

回到 [README](#) 確認自己已經完成開課前的三件事，再往下看

[02-環境設定.md](#) 熟悉第一天現場要做的環境設定步驟。

環境設定 | 開課當天現場動手建置

前提：[01-開課前準備.md](#) 的帳號都已經辦好。

這一頁是課堂第一節「環境建置」時照著做的步驟，做完環境就緒，能跑出第一支程式。

這不是雜務，是正式的教學單元：自己裝過一次 Python 與開發工具，

之後遇到環境問題才知道從哪裡查起。裝的時間算在課程裡，不是要壓縮的成本。

① 安裝 Python

1. 到 python.org/downloads，下載 Windows 版安裝檔。
2. 執行安裝檔時，**務必勾選「Add python.exe to PATH」**（畫面最下方的核取方塊）——漏勾這一步，之後在終端機打 `python` 會找不到指令。
3. 其餘選項用預設值，按「Install Now」完成安裝。

怎麼確認這步成功了：開啟終端機（開始功能表打「終端機」或 `cmd`），輸入：

```
python --version
```

看到版本號（例如 `Python 3.12.4`）就代表安裝成功、PATH 也設定對了。

② 安裝 Antigravity

1. 到 antigravity.google 下載 Windows 版安裝檔。
2. 執行安裝檔，跟著畫面指示完成安裝。
3. 開啟 Antigravity，用課前辦好的 Google 帳號登入。

怎麼確認這步成功了：登入後看得到 IDE 主畫面（左側檔案總管、下方終端機、右側或內建的 AI 助手面板），代表安裝與登入都成功。

備援方案：若 Antigravity 當機或裝不起來，改用 [Cursor](#) 安裝，操作方式與下面各步驟相同（都是 VS Code 系的 IDE）。

③ 安裝 Chrome

Antigravity 的登入與部分功能需要 Chrome 瀏覽器。

1. 到 google.com/chrome 下載並安裝。

怎麼確認這步成功了：開啟 Chrome，能正常瀏覽網頁。

④ 開終端機安裝套件

1. 在 Antigravity 裡，用選單「Terminal → New Terminal」（或快速鍵）開啟內建終端機。
2. 輸入以下指令，安裝這兩天實作會用到的套件：

```
pip install requests matplotlib
```

怎麼確認這步成功了：安裝過程沒有出現紅字錯誤，最後一行顯示

Successfully installed ...。可以再輸入 `pip list` 確認清單裡看得到 `requests` 與 `matplotlib`。

⑤ 在 IDE 裡 Clone GitHub 倉庫

1. 到 GitHub，右上角「+」→「New repository」，取名（例如 `tariff-semi-group3`），選擇 **Public**，勾選「Add a README file」，建立完成。
2. 進入倉庫頁面，點上方「Settings」→左側「Collaborators」→「Add people」，把組員的 GitHub 使用者名稱或信箱加進去；組員要**按接受邀請**才算真的加入。
3. 複製倉庫網址（綠色「Code」按鈕，複製 HTTPS 連結）。
4. 回到 Antigravity，用 Source Control 面板的「Clone Repository」（或指令面板打「Git: Clone」），貼上網址，選一個本機資料夾存放。

怎麼確認這步成功了：Antigravity 左側檔案總管看得到剛剛 clone 下來的倉庫檔案（至少有一個 `README.md`）；請組員也各自 clone 一次，確認看得到內容、也能編輯（不是唯讀）。

⑥ 跑出第一支程式，確認環境通了

1. 在 Antigravity 裡新增一個檔案 `hello.py`，內容：

```
python print("環境設定完成")
```
2. 在終端機輸入 `python hello.py` 執行。

怎麼確認這步成功了：終端機印出 `環境設定完成`，代表 Python、PATH、IDE 三件事都串起來了，可以正式開始寫程式。

Day 1 收尾要寫的兩份檔案

下課前，除了程式與 `PLAN.md` / `TASK.md` / `MEMORY.md` 要 push 之外，

再多寫兩份檔案，也一起存進倉庫：

- **SETUP.md**：把今天①～⑥的安裝步驟，改寫成「自己組的環境重建步驟」——裝了什麼、版本多少、`pip install` 了哪些套件。範本見 `templates/SETUP.md`。
- **HANDOFF.md**：今天做完什麼、明天第一步、目前卡在哪、環境有什麼特別設定。範本見 `templates/HANDOFF.md`。

為什麼要寫這兩份：教室電腦有還原系統，Day 2 開始前電腦會被清空——這代表 Day 2 一開場就要**照著自己昨天寫的 SETUP.md 重建環境**。重建順利，代表文件寫得夠好；卡住，代表文件漏了什麼，當場補上。這就是「環境可重現」的實地演練，也是交接能力的實測：寫給明天的自己看，看得懂嗎？

每過一關就 commit

這是這兩天最重要的習慣，沒有之一。

教室電腦有還原系統，下課會清空——存在電腦桌面或本機資料夾的東西，隔天就不見。只有存進 GitHub 倉庫的東西才是真的屬於自己的。所以：

- 完成五關卡（抓、理、看、監控化、結論報告）**任何一關**，就用 Antigravity 左側的 Source Control 面板存一次：填 commit 訊息 → Commit → Sync。
- 不要等到快下課才想起來存檔——commit 是遊戲裡的存檔點，越常存，風險越低。
- `PLAN.md`、`TASK.md`、`MEMORY.md` 三個檔案也是每次更新內容就存一次，不用等到寫完整份才存。

這個習慣的完整說明（為什麼、什麼時候做、下課前的收尾儀式）在 [03-開發流程六件套.md](#)，這是本課程最重要的一份文件，請務必花時間讀完。

開發流程六件套

這是本倉庫最重要的一份文件。兩天的實作，從頭到尾都在走這一套流程。
六件套：**GitHub 倉庫**、**push**、**定期 commit**、`PLAN.md`、`TASK.md`、`MEMORY.md`。

為什麼要有這一套

直接跟 AI 說「幫我寫一個抓股價的程式」，AI 只能用猜的猜你要什麼，猜錯了你也說不清楚哪裡不對。沒有寫清楚的規格，就沒有辦法驗收。這一套流程解決的就是這個問題：動手前先把要做的事講清楚，過程中把交辦跟卡點都記下來，事後才有東西可以查、可以交接。

拷問 → `PLAN.md` → `TASK.md` → `MEMORY.md` → 結論報告

- **拷問**：動手前先讓 AI 當面試官，把你還沒想清楚的地方問出來。
- **PLAN.md**：整個工具要做什麼、怎麼算做完——六節，開工前定案。
- **TASK.md**：現在這一步，交辦給 AI 的單子——目標、材料、驗收三行。
- **MEMORY.md**：做了什麼、卡在哪、AI 哪裡講錯、下一步——過程的紀錄，也是報告的素材。
- **結論報告**：最後讓 AI 讀自己抓到的資料，產出一頁輔助判斷的報告。

一、拷問式互動：動手前先被 AI 問倒

「拷問」的意思是：不要一開始就叫 AI 動手做，而是先請 AI 當一個嚴格的面試官，針對你的規劃一題一題問下去，問到你答不出來為止。**答不出來的地方，就是還沒想清楚的洞**，先在對話裡補完，再開始寫程式——比寫到一半才發現規劃有問題，成本低得多。

第一步：開新對話，先貼這段預載指令

跟 AI 開始任何一段新對話之前，先貼上這一段。它會讓 AI 先把你接下來輸入的話轉換成更精確的指令，再依照轉換後的指令回答。

在開始撰寫任何回覆之前，請主動執行以下步驟：

第一步：專家優化——你現在是一名深耕 AI 領域的顧問，專精於提示詞優化邏輯。

當我輸入「原始提示詞」時，請先將其轉化為更精確、能讓模型發揮最高效能的指令。

優化核心原則：最有效的提示詞平均約為 21 個字。若原始提示詞過於簡短，

請務必在轉化過程中強化「背景資訊」維度，以確保回覆品質。

轉化時必須包含以下四個維度：

〔角色任務〕定義專業身份與核心目標。

〔背景資訊〕提供必要的情境。

〔具體指令〕拆解明確的操作步驟。

〔約束條件〕規定字數、格式、語氣，且一律使用「正向表述」。

第二步：直接答覆——優化完成後，請立即依照該套優化後的指令進行答覆，產出最終內容。

請以「精準、聚焦、最小充分回答」為原則回應。

這段指令裡的四個維度——角色任務、背景資訊、具體指令、約束條件——正好就是後面

`TASK.md` 三行（目標、材料、驗收）的完整版，先看過一次，後面寫 `TASK.md` 會更有感覺。

第二步：拷問起手式

貼完預載指令之後，換一段新的訊息，把你目前的規劃講給 AI 聽，並請它開始拷問。

以下是一個完整範例，情境是抽到題 3（證交所盤後資料：台積電比大盤強還是弱）：

貼起手式，讓 AI 開始問

我們：我們要做的工具是：抓台積電與 0050 的日線，做成相對強弱監控。

請你當嚴格的面試官，一次問一題，拷問這份規劃：

目標、資料來源、做完的標準、可能失敗的地方。問到我們答不出來為止。

AI：第一題——這個工具做好之後，看的人要能回答什麼問題？

第三步：被問倒的地方，就是洞

我們：想看台積電走勢是不是比大盤強。

AI：第二題——「比大盤強」怎麼量化？看差幾個百分點、還是看比值？

我們：.....（答不出來）

AI：第三題——資料從哪裡來？那個來源的日期格式長什麼樣？

我們：證交所.....格式沒看過。（又一個洞）

被問倒不是壞事，是賺到——這兩個洞現在補，成本是幾分鐘；寫到一半才發現，成本是一個下午。把答不出來的問題記進 `MEMORY.md`，這就是要補的洞。

第四步：吵清楚了，命令 AI 產出 PLAN.md

跟組員討論、補完洞之後，回到對話裡把結論講給 AI，並且**明確要求輸出六節的**

`PLAN.md`——不是「幫我寫個計畫」這種空泛的要求，而是點名六節：

我們：洞補完了——比值定義用「台積電收盤÷0050 收盤」，資料源是證交所月報表、民國年要轉西元。請把以上討論整理成 PLAN.md，六節：

①目標 ②資料（來源網址、格式、已知的坑）③功能清單（拆步驟）④驗收條件（可核對）⑤除錯線索 ⑥維護方式。存檔用的 Markdown 給我。

拿到 AI 產出的 PLAN.md 之後，存進倉庫（見 [templates/PLAN.md](#) 的六節範本）。

二、許願式 vs 交辦式

PLAN.md 定案之後，接下來每一步要請 AI 動手做事，都要用 TASK.md 交辦，而不是隨口許願。兩者差在哪：

	許願式	交辦式
講法	「幫我寫一個抓股價的程式」	目標、材料、驗收三行寫清楚
AI 拿到的資訊	幾乎沒有，只能用猜的	知道要做什麼、能用什麼、怎麼算做完
結果	通常要來回猜好幾輪才對到需求	一次就給得出能用的東西，即使不對也容易指出哪裡錯

每開始一步，先寫 TASK.md 再貼給 AI——這是這兩天最需要養成的習慣。

範本與填寫範例見 [templates/TASK.md](#)。

AI 助手的位置：AI 就在 IDE 裡，不必把 TASK.md 的內容複製貼上——直接跟它說「讀 TASK.md 然後照著做」，它能直接開啟並讀取專案裡的檔案。這樣比開瀏覽器問、再貼回來省時，也更省免費額度。

三、每過一關就 commit

時機：完成五關卡（抓、理、看、監控化、結論報告）任何一關，或是 PLAN.md / TASK.md / MEMORY.md 有更新，就存一次。不要等到快下課才想起來。

方法：在 Antigravity（或 Cursor）左側的 **Source Control 面板** 操作：填 commit 訊息 → 按「Commit」→ 按「Sync」。左側面板會直接顯示哪幾行改了，這是比網頁上傳更直覺的地方——改了什麼、要存什麼，一眼就看到。

為什麼這麼重要：教室電腦有還原系統，下課就清空。commit 是遊戲裡的存檔點——存得越勤，風險越低；沒存回倉庫，等於白做。

四、下課前的收尾儀式

每天下課前十分鐘，做完這四件事再離開：

1. **程式 + 三個檔案全部存回倉庫（push）**——確認 GitHub 網頁上看得到最新版本。

2. **MEMORY.md** 最後寫「明天第一步」——不用寫長，一句話讓明天的自己（或隊友）一看就知道從哪裡接手。寫法範例見 [templates/MEMORY.md](#)。
3. 寫 **SETUP.md** 與 **HANDOFF.md** 並存回倉庫——**SETUP.md** 記今天怎麼把環境裝起來（範本見 [templates/SETUP.md](#)），**HANDOFF.md** 記今天做完什麼、卡在哪、環境有什麼特別設定（範本見 [templates/HANDOFF.md](#)）。教室電腦 Day 2 會被清空，這兩份是重建環境的依據。
4. **確認組員都能存取倉庫**——如果有人臨時加入或還沒收到邀請，這時候補上。

第二天開場，第一件事就是**照著自己的 SETUP.md 重建環境**，重建順利代表文件寫得夠好，卡住就代表文件漏了什麼，當場補上；環境通了之後，把 **HANDOFF.md** / **MEMORY.md** 讀給 IDE 裡的 AI 助手，接著往下做——交接寫得好，開場十分鐘就能熱機完成，不用重新想昨天在哪裡卡住。

五、六件套如何對映到五關卡

六件套是「怎麼跟 AI 一起工作」的流程，五關卡（抓→理→看→監控化→結論報告，見 [tasks/README.md](#)）是「工具本身要做出什麼」。

兩者疊在一起看：

- **PLAN.md** 的「功能清單」那一節，內容就是五關卡的拆解版。
- 每進到下一關，就開一張新的 **TASK.md** 交辦。
- 每過一關或卡住，就在 **MEMORY.md** 記一筆。
- 最後一關「結論報告」，本身也是用一張 **TASK.md** 交辦（目標：讀自己的資料產出四節報告），細節見 [reference/結論報告格式.md](#)。

程式會過時，這一套流程帶得走——下一個專案，同一套流程。

PLAN.md 範本

用途：整個工具要做什麼、怎麼算做完。在「拷問」被問倒、把洞補完之後，命令 AI 產出這份文件，六節一次到齊，存進自己的倉庫。
寫法背景見 [../docs/03-開發流程六件套.md](#) 第一節。

填寫範例（題 3：證交所盤後資料爬蟲，台積電×大盤）

① 目標

自動抓台積電（2330）與 0050 的日線，算出兩者的相對強弱比值，做成重跑一次就能更新到最新交易日的監控工具，並在圖上標出關稅事件日。

② 資料（來源網址、格式、已知的坑）

- 來源：證交所 **STOCK_DAY**，https://www.twse.com.tw/exchangeReport/STOCK_DAY?response=json&date=YYYYMMDD&stockNo=2330

- 格式：JSON，`data` 為二維陣列，日期欄位是民國年（例如 `114/07/01`）
- 已知的坑：一次只回一個月，要抓一整年得跑 12 次；數字欄位有千分位逗號要清除；連續抓取要加延遲，避免被限速

③ 功能清單（拆步驟）

1. 抓：逐月呼叫 API，把台積電與 0050 的原始 JSON 各自存成 CSV
2. 理：民國年轉西元、去除千分位逗號、兩張表用日期對齊
3. 看：算比值（台積電收盤 ÷ 0050 收盤），畫成折線圖，標上至少 2 個關稅事件日
4. 監控化：整理成一支腳本，重跑會自動補到最新交易日
5. 結論報告：把最新比值與圖表餵給 AI，產出四節結論報告

④ 驗收條件（可核對）

- 原始資料存檔，說得出網址與抓取時間
- 隨機抽一天，比值算出來的數字跟證交所網站對得上
- 圖上事件日清楚可指
- 台上現場重跑一次成功

⑤ 除錯線索

- 抓不到資料：先檢查日期格式是不是民國年、`stockNo` 有沒有打對
- 比值算出來是 `NaN`：多半是兩張表的日期沒對齊，中間有缺值
- 圖表事件日標錯位置：檢查日期是不是西元、格式是不是一致

⑥ 維護方式

之後每次要更新到最新資料，重跑整支腳本即可，不用手動改程式碼；
如果證交所改版式，先看 `STOCK_DAY` 回傳的 JSON 結構有沒有變。

空白版（複製這一段填寫）

① 目標

② 資料（來源網址、格式、已知的坑）

③ 功能清單（拆步驟）

④ 驗收條件（可核對）

⑤ 除錯線索

⑥ 維護方式

TASK.md 範本

用途：現在這一步，交辦給 AI 的單子。三要素——**目標、材料、驗收**。

每開始一步，先寫這三行再貼給 AI，不要用一句話許願。

許願式 vs 交辦式的對照見 [../docs/03-開發流程六件套.md](#) 第二節。

填寫範例（承接 PLAN.md 範例，第一張任務單）

目標：抓 2025 全年台股電（2330）日線

材料：

- PLAN.md 第②節的端點與參數
(https://www.twse.com.tw/exchangeReport/STOCK_DAY?response=json&date=YYYYMMDD&stockNo=2330)
- 已知的坑：一次只回一個月、日期是民國年、數字欄位有千分位逗號

驗收：

- 存成 CSV，日期欄位已轉成西元格式
- 12 個月都抓到，沒有缺月
- 隨機抽一天，跟證交所網站上顯示的收盤價對得上

空白版（複製這一段填寫）

目標：

材料：

- 1.
- 2.

驗收：

- 1.
- 2.
- 3.

提醒

- 材料盡量具體到「貼哪一段」，例如「把 `PLAN.md` 第②節整段貼上」，不要只寫「參考 PLAN」讓 AI 自己回頭找。
- 驗收條件要能核對，不要寫「看起來對就好」這種無法檢查的標準。
- 拿到 AI 的產出後，照驗收條件一條一條核對，不符合的就在同一個對話裡指出來，不用重新開一段。

MEMORY.md 範本

用途：做了什麼、卡在哪怎麼繞過去、AI 哪裡講錯怎麼發現、下一步——四行。

每次卡關或過關就記一筆，不要等到最後才補寫。兩份收穫：

- ①斷線、重開對話、隔夜接手時，把這份貼給新對話就能接上；
- ②分組報告的「過程與困難」「解決辦法」，台上不用編，這裡打開就是。

填寫範例（一整天下來的記錄，由上到下累加）

2026-08-18 09:40

做了什麼：PLAN.md 完成，六節都寫好存進倉庫

卡點：比值定義（差百分點還是比值）吵了 10 分鐘

AI 哪裡講錯：（目前沒有）

下一步：TASK 第一張—抓台積電 2025 全年日線

2026-08-18 10:15

做了什麼：抓資料的程式寫好，跑出 12 個月的 CSV

卡點：8 月抓不到資料，後來發現是日期參數少補零（8 要打成 08）

AI 哪裡講錯：AI 一開始說證交所日期參數可以用 8 位數任意格式，

實測後發現月份與日期一定要補零成兩位數，是 AI 講錯的地方

下一步：把台積電與 0050 兩份資料做日期對齊，清資料

2026-08-18 11:50（收尾）

做了什麼：資料清洗完成、對齊，數值都能算了

卡點：0050 有一天缺值（當天無成交），先用前一日補值處理

AI 哪裡講錯：無

下一步（明天第一步）：把比值畫成折線圖，標關稅事件日；

事件日清單已經列在 PLAN.md 第③節備註，明天直接抓來用

空白版（每次卡關或過關，複製這一段接著寫）

日期 時間

做了什麼：

卡點：

AI 哪裡講錯：

下一步：

下課前的收尾寫法

下課前最後一筆記錄，「下一步」要寫成「明天第一步」，具體到看了就能動手，不要寫「繼續做」這種空泛的話。好的寫法像上面範例最後一筆：說清楚要做什么、需要的材料在哪裡找。

SETUP.md 範本

用途：環境重建步驟——從一台空白電腦到能跑出你們的程式，一步一步寫下來。

Day 1 收尾時寫，Day 2 開場照著自己寫的這份重建環境。重建順利，代表寫得夠好；卡住，代表漏了什麼，當場補上再繼續。

完整安裝步驟見 [../docs/02-環境設定.md](#)，這裡只記

自己這組實際裝了什麼、版本多少，不用重抄一遍教學文。

填寫範例

環境重建步驟（第 3 組）

1. 安裝 Python 3.12 (python.org 下載，安裝時勾選 Add to PATH)

確認：終端機打 `python --version` 看到 `Python 3.12.4`

2. 安裝 Antigravity (antigravity.google 下載，用 Google 帳號登入)

確認：開啟後左側檔案總管、下方終端機都看得到

3. 安裝 Chrome (Antigravity 登入需要)

確認：能正常開網頁

4. 終端機安裝套件：

```
pip install requests matplotlib
```

確認：`pip list` 看得到 requests 2.32.x、matplotlib 3.9.x

5. Clone 倉庫：

在 Antigravity 用 Source Control 面板 Clone Repository，

貼上 <https://github.com/xxx/tariff-semi-group3>

確認：左側檔案總管看得到 README.md、PLAN.md

6. 跑第一支程式確認：

```
python probes/fetch_twse.py
```

確認：終端機印出 2330 最新一筆收盤價，沒有紅字錯誤

這組的特別設定

- pip 安裝很慢時，改用 `pip install -i https://pypi.org/simple requests matplotlib`

- Windows 中文路徑跑程式偶爾會亂碼，統一在終端機打 `chcp 65001` 先切成 UTF-8

空白版（複製這一段填寫）

環境重建步驟（第__組）

1. 安裝 Python（版本：__）

確認：

2. 安裝 Antigravity（或 Cursor）

確認：

3. 安裝 Chrome

確認：

4. 終端機安裝套件：

確認：

5. Clone 倉庫：

網址：

確認：

6. 跑第一支程式確認：

指令：

確認：

這組的特別設定

-

提醒

- 每一步都要寫「怎麼確認這步成功了」，不是「裝完就好」——Day 2 卡住的時候，就是靠這一行判斷卡在哪一步。
- 套件版本盡量寫具體數字（`pip list` 或 `pip freeze` 可以看到），不要只寫「裝了 requests」。
- 如果這組有特別的環境問題（例如某台電腦裝不起來、要換別的下載來源），寫進「這組的特別設定」，明天照著避開同一個坑。

HANDOFF.md 範本

用途：進度交接——今天做完什麼、明天第一步、目前卡在哪、環境有什麼特別設定。
Day 1 收尾寫一次，Day 2 開場（重建完環境後）第一件事就是把這份讀給 AI 助手，
接著往下做。跟 MEMORY.md 的差別：MEMORY.md 是逐筆的過程記錄，
HANDOFF.md 是收尾時整理出來的「一頁重點」，方便明天快速接手，不用整份翻。

填寫範例

```
# HANDOFF.md (第 3 組，2026-08-18 收工)

## 今天做完什麼
- PLAN.md 六節定案
- 關卡一、二完成：抓到台積電與 0050 全年日線，清洗對齊完成
- 資料存在 raw/ (原始 JSON) 與 data/aligned.csv (清洗後)

## 明天第一步
- 照 SETUP.md 重建環境
- 用 data/aligned.csv 畫圖，標六個關稅事件日（清單在 PLAN.md 第③節備註）

## 目前卡在哪
- 事件日標籤還沒想好怎麼排版（六條線文字會擠在一起），明天要先解決這個再往下

## 環境有什麼特別設定
- pip 安裝套件時教室網路較慢，改用 `--https://pypi.org/simple` 參數
- 詳細安裝步驟見 SETUP.md，這裡不重複
```

空白版（複製這一段填寫）

```
# HANDOFF.md (第__組，__ 收工)

## 今天做完什麼
-

## 明天第一步
-

## 目前卡在哪
-

## 環境有什麼特別設定
-
```

提醒

- 「明天第一步」要具體到看了就能動手，不要寫「繼續做」這種空泛的話。
- 「目前卡在哪」誠實寫，這也是分組報告「過程與困難」的素材，不用另外編。
- Day 2 開場把這份連同 `MEMORY.md` 一起讀給 IDE 裡的 AI 助手（直接說「讀 `HANDOFF.md` 和 `MEMORY.md`，接著往下做」），比複製貼上省時間。

結論報告格式

每一題的終點不是圖表，是**決策輔助**：把工具抓到、算出來的東西，連同 `PLAN.md` 一起交給 AI，產出一頁結論報告，輔助自己下判斷。這是五關卡的最後一關（見 [../tasks/README.md](#)），用一張 `TASK.md`（範本見 [../templates/TASK.md](#)）交辦即可。

做法

工具跑出最新結果後，把輸出（數字摘要 + 圖）連同 `PLAN.md` 一起餵給 AI，要求產出一頁結論報告，四節：

1. **現況數字**——引用自己的資料，寫出目前看到的具體數字。
2. **與關稅事件日的對照**——這些數字在關稅事件日前後有什麼變化。
3. **判讀**——這些數字說明了什麼。
4. **限制與失效條件**——哪裡不能信、什麼情況下這個判讀會翻盤。

三條驗收

1. 報告裡的每一個數字都能在自己抓到的資料裡指出來（不是 AI 憑空生成的數字）。
2. 第④節「限制與失效條件」不能空白。
3. 分組報告時展示這份結論報告，並口頭講一分鐘。

合規框定

- 結論報告是「輔助自己判斷」的整理，**不是投資建議**。
- **不寫買賣指令、不寫目標價**。
- 第④節（限制與失效）是必要條款——這也是 [四判準自檢.md](#) 的落地：一份經得起檢驗的判讀，一定說得出自己在什麼情況下會失效。

怎麼揪出 AI 瞎編的數字

拿到 AI 產出的報告後，逐一核對第①節的數字：能不能在自己的 CSV 或圖表裡找到同一個數字？找不到，就是 AI 引用了自己資料裡沒有的東西，要請 AI 重新核對後改寫。這個核對動作本身就是報告展示時值得講的一段——「AI 哪裡講錯、我們怎麼發現的」。

空白模板（複製這一段填寫）

結論報告

① 現況數字

② 與關稅事件日的對照

③ 判讀

④ 限制與失效條件

四判準自檢

演講段教的四個判準，用在檢驗任何一句「A 造成了 B」的說法站不站得住腳。

Day 2 開場會回扣這四條，也是結論報告第④節「限制與失效條件」的落地標準——寫報告前先拿這四條問自己一遍。

① 時間順序對嗎

怎麼問自己：說「事件在先、反應在後」的那句話，時間點真的對得上嗎？

資料上的日期是不是真的排在事件日之後，不是自己記錯或看反了。

答不出來時該怎麼調整說法：如果時間順序模糊（例如新聞稿發布時間跟市場反應幾乎同一天，分不出先後），說法要改成「同時發生」而不是「因為 A 所以 B」，不要硬套因果關係。

② 有沒有別的原因

怎麼問自己：這段時間除了關稅事件之外，還有沒有別的事也在同一段時間發生（例如財報公布、其他總體經濟消息、產業本身的循環），有沒有可能是那個原因，不是關稅事件造成的。

答不出來時該怎麼調整說法：說不出排除其他原因的理由時，判讀要加上但書，例如「這段期間同時有其他消息，無法排除是別的原因造成」，不要把單一事件講成唯一原因。

③ 換一個對象還成立嗎

怎麼問自己：如果換一檔股票、換一個商品、換一段類似的時間，這個現象還會一樣出現嗎？還是只有這一次、這一檔剛好對上。

答不出來時該怎麼調整說法：沒有多換幾個對象驗證過，就不要說「這是普遍現象」，改成「在我們看的這一個樣本裡觀察到」，把結論限縮到自己實際檢驗過的範圍。

④ 只發生一次還是每次都這樣

怎麼問自己：拉長時間看，類似的事件過去發生過幾次？每一次都出現同樣的反應，還是這次剛好而已（樣本數只有一兩個事件日）。

答不出來時該怎麼調整說法：樣本數少的時候（例如題 9 事件日反應統計，入統計的事件日通常只有個位數），結論的信心要降低，講法從「這個規律成立」改成「目前觀察到的幾次事件裡是這樣，樣本數不足以下定論」。

用在結論報告第④節的寫法

第④節「限制與失效條件」不是隨便寫一句「僅供參考」就過關，而是要具體回答：用這四條問過一輪之後，自己的判讀在哪一條上答不出來、因此要怎麼降低講法的信心。分組報告展示結論報告時，能不能講出這一段，就是講評的重點。

資料源速查 | 六個資料源

六個資料源、各自的用途、端點、回傳格式與已知坑，統整自課前實測結果。探測腳本與樣本檔案在 [../probes/](#)，可以直接參考改寫。任一資料源當天連不上，先跟講師拿備援檔起步，課後再自己重抓。

1. 證交所 STOCK_DAY (個股日成交)

用途：抓個股或 ETF 的歷史日線（開高低收、成交量）。

端點：

```
https://www.twse.com.tw/exchangeReport/STOCK_DAY?response=json&date=YYYYMMDD&stockNo=2330
```

date 只要是該月任一天即可，**stockNo** 換成想抓的證券代號（例如 0050）。

回傳格式：JSON，**data** 為二維陣列，欄位順序見同一份回應裡的 **fields**：

日期 / 成交股數 / 成交金額 / 開盤價 / 最高價 / 最低價 / 收盤價 / 漲跌價差 / 成交筆數 / 註記。

已知坑：

1. **一次只回一個月**，要抓一整年得跑 12 次。
2. **日期是民國年格式**（如 114/07/01），要自己轉西元。
3. 數字欄位有千分位逗號，要清除才能轉成數值。
4. 有隱性限速，連續抓取要加延遲（每次 1-2 秒）。

對應題號：題 3、9、10。

2. 證交所 OpenAPI (openapi.twse.com.tw)

用途：抓當日全市場的個股資訊或大盤指數統計，不用逐檔查詢。

端點：

```
GET https://openapi.twse.com.tw/v1/exchangeReport/STOCK_DAY_ALL (個股日成交資訊，全市場約 1300+ 檔)
GET https://openapi.twse.com.tw/v1/exchangeReport/MI_INDEX (大盤與各類股指數統計，約 260+ 種指數)
```

回傳格式：JSON 陣列。STOCK_DAY_ALL 欄位為英文縮寫

(Date / Code / Name / TradeVolume / TradeValue / OpeningPrice / HighestPrice / LowestPrice / ClosingPrice / Change / Transaction) ; MI_INDEX 欄位為中文
(日期 / 指數 / 收盤指數 / 漲跌 / 漲跌點數 / 漲跌百分比 / 特殊處理註記) 。

已知坑：

1. **欄位名稱是英文縮寫**，要對照文件才知道對應到什麼 (TradeVolume 是成交股數，TradeValue 是成交金額，容易搞混) 。
2. **日期是民國年純數字字串** (如 "1150814")，要自己轉西元、切年月日。
3. **這兩個端點都只回傳「當日」一筆快照**，沒有 start/end 日期參數；要做歷史監控表，得每天各存一份、自己累積歷史，不能像 STOCK_DAY 那樣一次查一個月。
4. 官方 OpenAPI 目前查無三大法人買賣超端點。
5. 建議加上 User-Agent 標頭，沒有寫有時會被視為異常流量。

對應題號：題 4。

3. 期交所盤後資訊 (每日行情)

用途：抓期貨 (如台指期) 每日的盤後行情。

端點：這是一個查詢頁面，用 POST 帶查詢參數，不是單純的 GET 網址：

```
POST https://www.taifex.com.tw/cht/3/futDailyMarketReport
Content-Type: application/x-www-form-urlencoded

queryType=2
marketCode=1
commodity_id=TX          # 台指期，可換其他契約代碼
queryDate=YYYY/MM/DD
```

回傳格式：整頁 HTML，資料在其中一個 <table> 內，沒有現成的 CSV / JSON 下載連結。

欄位 (依表格順序)：契約 / 到期月份(週別) / 開盤價 / 最高價 / 最低價 / 最後成交價 / 漲跌價 / 漲跌% / 成交量 / 結算價 / 未沖銷契約量 / 最佳買價 / 最佳賣價 / 歷史最高價 / 歷史最低價。

已知坑：

1. **這是表單 POST**，要用程式送 POST body，不能直接 GET 網址。

2. **HTML 標籤大小寫混用**：同一份 HTML 裡 `<td>` 與 `<TD>` 混用，用正則解析一定要加 `IGNORECASE`，否則後面欄位會整排錯位。
3. **表格第一列是「近月契約」**，後面還有次月、季月等多筆，抓資料前要先確認要哪一列。
4. **假日或國定假日無資料**，回傳頁面沒有資料表格，程式要判斷「查無資料」並跳過。
5. 此頁「**結算價**」「**未沖銷契約量**」固定回傳 `-`，不是真的資料，要拿到這兩欄需另外找期交所別的報表頁面。
6. **近月契約會換月**（例如 202606 → 202608），直接串成連續價格序列會在換月日出現跳空，要自己處理換月或固定只取單一月份契約。

對應題號：題 5。

4. Google News RSS

用途：查關鍵字相關新聞，整理成時間軸或摘要。

端點：

```
https://news.google.com/rss/search?q=<關鍵字, URL-encode>&hl=zh-TW&gl=TW&ceid=TW:zh-Hant
```

回傳格式：標準 RSS 2.0 XML，`<item>` 內含 `title`、`link`、`pubDate`、`source`，UTF-8 編碼，中文無亂碼。單次查詢固定回傳約 100 筆。

已知坑：

1. **link 是 Google 轉址網址**，不是原始新聞網址，開啟後才會跳轉到真正來源；直接存的話，日後原站若下架連結仍可能失效。
2. **標題重複**：同一事件被多家媒體報導，標題高度相似或完全相同，需要用標題模糊比對或連結去重。
3. **pubDate 是 UTC 時間**，不是台北時間，畫時間軸前要 +8 小時。
4. **關鍵字組合影響很大**：空白代表 AND，加引號代表精準片語，結果差很多，建議在 `PLAN.md` 記錄關鍵字怎麼選、為什麼。
5. 沒有分頁機制，一次查詢就是全部結果，抓不到「更舊」的新聞，只能靠關鍵字或時間詞（如加 `after:2026-08-01`）調整範圍。

對應題號：題 1、2。

5. Yahoo 美股 chart 端點

用途：抓美股個股或外匯的歷史日線。

端點：

```
https://query1.finance.yahoo.com/v8/finance/chart/<代號>?range=1y&interval=1d
```

股票代號直接用代碼（如 `NVDA`），外匯代號用 `<幣別>=X`（如 `JPY=X` 代表 USD/JPY）。

回傳格式：JSON，`chart.result[0]` 內含 `meta`（含 `longName` 名稱）、`timestamp`（Unix 秒數陣列，UTC）、`indicators.quote[0]` 下的 `open` / `high` / `low` / `close` / `volume`。

已知坑：

1. **非官方端點**，沒有正式公開文件保證長期穩定，格式未來可能變。
2. **時區用 UTC timestamp 自己換算**，不要直接信任回傳裡 `meta` 顯示的時區名稱（外匯資料的時區欄位有時顯示的地區跟報價來源無關）。
3. 美股收盤日期與台灣時間有時差認知問題，串接台股資料前要弄清楚拿到的是美國當地哪一天。
4. 外匯資料**只有開高低收（中價）**，**沒有買入 / 賣出雙向報價**，如果題目需要買賣雙價，這個端點無法提供。

對應題號：題 7、8、9、10。

6. 群益交易王 CTA（外匯與指數）

用途：取得外匯與海外指數的即時 / 歷史報價，或在平台上用 EvoCode（C#）寫自訂指標。

取得方式：不是網路 API 端點，而是安裝群益交易王電腦版軟體（安裝步驟見

[../docs/02-環境設定.md](#) ④），用模擬帳號登入後在軟體介面裡

掛商品圖表，或在 EvoCode 開發環境裡透過 `HistoryDataSeries` / `InstrumentsManager` 等

API 存取報價（見 [EvoCode速查.md](#)）。

回傳格式：平台圖表報價（可另存為資料檔）或 EvoCode 指標讀到的數值序列，沒有固定的 JSON / CSV 規格，依平台介面操作為主。

已知坑：

1. **CFD 報價有買價 / 賣價價差（spread）**，用哪一個要在報告裡寫清楚。
2. **各商品交易時間不同**，恆生指數與其他海外指數的時段差很多，動手前先確認。
3. **模擬帳號需先申請並收信取得帳號**，申請流程見 [../docs/01-開課前準備.md](#)，務必課前辦好。
4. **沒有新台幣（TWD）貨幣對、沒有美元指數（DXY）、沒有費城半導體指數（SOX）**，詳見 [../tasks/題06.md](#) 的三個缺口說明。

對應題號：題 6（含題 9、10 選擇 EvoCode 路徑的組別）。

7. 新台幣匯率與美元指數（公開資料源）

用途：群益 CTA 平台沒有新台幣貨幣對，也沒有美元指數。要看這兩個，用這裡的公開來源。

對應題號：題 6 的補充；也是演講段提到台幣急升與美元走弱時，可以自己去做驗證的地方。

端點：Yahoo chart 端點（與第 5 節同一套寫法，只是換代號）

```
https://query1.finance.yahoo.com/v8/finance/chart/{代號}?range=1y&interval=1d
```


代號	內容	實測 (2026-08-17)
TWD=X	美元兌新台幣	✅ 近一年 263 筆
^NYICDX	ICE 美元指數	✅ 近一年 251 筆， 無空值，優先用這個
DX-Y.NYB	同一個美元指數的另一個代號	⚠️ 可用但有 52 個空值，不建議
JPY=X / KRW=X	美元兌日圓 / 韓元	✅ 各 263 筆，可作對照

已知坑

1. **只有中價**，沒有買入 / 賣出雙向報價——報告要寫明用的是中價。
2. 非官方端點，格式可能被改，不要假設永遠可用。
3. **最後一筆常是當下的即時快照**，不是完整交易日，畫圖前先判斷要不要剔除。
4. 台灣銀行牌告匯率 (`rate.bot.com.tw`) 會被機器人偵測攔下，需要瀏覽器執行 JavaScript 才能通過。本課程不繞過這類保護機制，因此不列為可用路徑；若一定要用台銀的雙向報價，請自行到網站上手動下載。

現成腳本： `../probes/probe-fx.py` (四個代號一次抓完，輸出到 `probes/output/`)